

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

递归与递推

Recursion and Recurrence: From Definition to Implementation

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 18 日

目录

1. 递归基础
2. 递推基础
3. 对比与进阶
4. 总结



目录

1 递归基础

2 递推基础

3 对比与进阶

4 总结

什么是递归

递归：一个函数直接或间接地调用自身，把大问题分解为同类型的小问题。

生活类比

- 两面镜子相对而放，看到无限层叠的影像
- 俄罗斯套娃，每一层都长得像上一层
- 查字典：查 A 词，解释里又有 B 词，再查 B 词

递归三要素

1. 边界条件：最小规模时直接给出答案，不再递归
2. 递归关系：大问题如何化为同类型的小问题
3. 状态变化：每次调用都必须向边界靠拢，否则死循环

$$\text{递归} = \text{边界} + \text{递推关系} + \text{向边界靠拢}$$

递归的执行过程：以阶乘为例

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

函数定义

```
1 long long fact(int n)
2 {
3     if (n == 0 || n == 1) return 1;   // 边界
4     return 1LL * n * fact(n - 1);     // 递归
5 }
```

调用 fact(4) 的两个阶段

向下展开

- fact(4) 需要 fact(3)
- fact(3) 需要 fact(2)
- fact(2) 需要 fact(1)
- fact(1) 命中边界，返回 1

向上回代

- fact(1) = 1
- fact(2) = 2 * 1 = 2
- fact(3) = 3 * 2 = 6
- fact(4) = 4 * 6 = 24

递归的执行过程：以阶乘为例

调用栈：每进入一层就压栈，每返回一层就出栈，栈深度等于递归次数。

例题：求阶乘

题目：给定非负整数 n，计算 $n! = 1 \times 2 \times \cdots \times n$，约定 $0! = 1$。

示例

- $n = 0$ 时，答案为 1
- $n = 1$ 时，答案为 1
- $n = 5$ 时，答案为 120
- $n = 10$ 时，答案为 3628800

求阶乘：解法

四步法

- **边界**：$n = 0$ **或** $n = 1$ **时返回** 1
- **递归**：$n! = n \times (n-1)!$
- **状态变化**：n **每次减** 1，**必定到达边界**
- **返回类型**：n **较大时结果超过** int，**用** long long

为什么这样递归：$n!$ 的定义本身就包含 $(n-1)!$，递归关系直接映射定义，不需要额外推导。

参考代码

求阶乘：解法


```
1 long long fact(int n)
2 {
3     if (n == 0 || n == 1)
4         return 1;
5     return 1LL * n * fact(n - 1);
6 }
```

注意：`1LL * n * ...` 先转 long long，避免中间结果按 int 计算溢出。

例题：斐波那契数列

题目：斐波那契数列定义如下，求第 n 项。

$$F(1) = 1,\ F(2) = 1,\ F(n) = F(n-1) + F(n-2)\ \ (n \geq 3)$$

示例

- $n = 1$ 时，答案为 1
- $n = 5$ 时，答案为 5
- $n = 10$ 时，答案为 55
- $n = 40$ 时，答案为 102334155

斐波那契：递归解法

四步法

- **边界**：$n = 1$ 或 $n = 2$ 时返回 1
- **递归**：$F(n) = F(n-1) + F(n-2)$
- **状态变化**：n 每次减 1 或减 2，向 1 与 2 靠拢
- **返回类型**：n 较大时用 long long

参考代码

```
long long fib(int n)
{
    if (n == 1 || n == 2)
        return 1;
    return fib(n - 1) + fib(n - 2);
}
```

斐波那契：递归解法

效率问题：$F(n)$ 会被计算多次。例如 $F(5)$ 会调用 $F(3)$ 两次、$F(2)$ 三次。递归树呈指数级展开，复杂度约为 $O(2^n)$。

定义贴近递归，但直接递归可能大量重复计算

例题：汉诺塔

题目：有三根柱子 A、B、C，n 个大小不同的盘子从上到下由小到大叠放在 A 上。要求把所有盘子移到 C，规则如下：

- 每次只能移动一个盘子
- 大盘子不能压在小盘子上
- 输出每一步的移动过程与总步数

示例

- $n = 1$ 时，移动 A 到 C，共 1 步
- $n = 2$ 时，A 到 B、A 到 C、B 到 C，共 3 步
- $n = 3$ 时，共 7 步
- $n = 4$ 时，共 15 步

汉诺塔：解法

分解思路：把 n 个盘子从 A 移到 C，借助 B。

- 第一步：把上面 $n-1$ 个盘子从 A 移到 B，借助 C
- 第二步：把最大的盘子从 A 移到 C
- 第三步：把 $n-1$ 个盘子从 B 移到 C，借助 A

三要素

- 边界：$n=1$ 时直接 A 到 C
- 递归：两次调用 $n-1$ 的子问题
- 状态变化：n 每次减 1，起始柱、辅助柱、目标柱在两次调用中交换

参考代码

汉诺塔：解法


```cpp
void hanoi(int n, char from, char via, char to)
{
    if (n == 1)
    {
        cout << from << " -> " << to << endl;
        return;
    }
    hanoi(n - 1, from, to, via);
    cout << from << " -> " << to << endl;
    hanoi(n - 1, via, from, to);
}
```

总步数：$T(n) = 2T(n-1) + 1$，**解为** $T(n) = 2^n - 1$。

递归的优缺点

优点

- 代码贴近问题定义，简洁易读
- 天然适合处理嵌套结构：树、括号、分解问题
- 分解思路清晰，便于口头推导

缺点

- 函数调用有开销，递归层数多时明显变慢
- 调用栈有空间上限，深度过大会栈溢出
- 朴素递归可能大量重复计算，效率急剧下降
- 调试不便，需要跟随调用栈逐层分析

递归是把问题变小，不是把问题变快

目录

1 递归基础

2 递推基础

3 对比与进阶

4 总结

什么是递推

递推：从已知的初始值出发，按固定规则逐步推出后续每一项，直到得到目标。

与递归的方向对比

- 递归：从大问题往回拆，拆到边界，再一路返回
- 递推：从小问题往前填，填到目标，直接得到答案

递推三要素

1. 状态定义：$dp[i]$ 表示什么
2. 转移方程：$dp[i]$ 由哪些前驱状态推出
3. 边界条件：最小规模时 dp 的值

递推顺序：保证计算 $dp[i]$ 时，它依赖的所有状态都已算好。

$$\text{递推} = \text{状态} + \text{转移} + \text{边界} + \text{顺序}$$

例题：爬楼梯

题目：有 n 级台阶，每次可以走 1 级或 2 级，问从第 0 级走到第 n 级共有多少种走法。

示例

- $n = 1$ 时，答案为 1
- $n = 2$ 时，答案为 2
- $n = 3$ 时，答案为 3
- $n = 4$ 时，答案为 5
- $n = 5$ 时，答案为 8

爬楼梯：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- 状态：$dp[i]$ 表示走到第 i 级的走法数
- 转移：$dp[i] = dp[i-1] + dp[i-2]$
- 边界：$dp[1] = 1$，$dp[2] = 2$
- 顺序：i 从 3 到 n 依次递推

为什么这样转移

- 走到第 i 级的最后一步只有两种来源：从 $i-1$ 走 1 级，或从 $i-2$ 走 2 级
- 这两类走法不重不漏，加起来就是 $dp[i]$

参考代码

爬楼梯：解法


```
int dp[Maxn + 5];
dp[1] = 1;
dp[2] = 2;
for (int i = 3; i <= n; i++)
    dp[i] = dp[i - 1] + dp[i - 2];
cout << dp[n] << endl;
```

注意：$n = 1$ 时若数组已写 $dp[2]$，会越界，需提前判断。

例题：杨辉三角

题目：输出杨辉三角的第 1 行到第 n 行。第 i 行有 i 个数，首尾均为 1，中间每个数等于上一行相邻两数之和。

示例：$n = 4$ 时的输出

- 第 1 行：1
- 第 2 行：1 1
- 第 3 行：1 2 1
- 第 4 行：1 3 3 1

示例：$n = 5$ 时的输出

- 第 5 行：1 4 6 4 1

杨辉三角：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- **状态**：$dp[i][j]$ 表示第 i 行第 j 列的数
- **转移**：$dp[i][j] = dp[i-1][j-1] + dp[i-1][j]$
- **边界**：$dp[i][1] = 1$，$dp[i][i] = 1$
- **顺序**：i 从 1 到 n，j 从 1 到 i

为什么这样转移：每个位置等于左上角加右上角，正是上一行相邻两数之和。

参考代码

杨辉三角：解法


```
int dp[Maxn + 5][Maxn + 5];
for (int i = 1; i <= n; i++)
{
    dp[i][1] = 1;
    dp[i][i] = 1;
    for (int j = 2; j < i; j++)
        dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
}
```

复杂度：时间 $O(n^2)$，空间 $O(n^2)$。

扩展：若只需最后一行，可用滚动数组把空间降到 $O(n)$。

目录

1 递归基础

2 递推基础

3 对比与进阶

4 总结

递归与递推对比

维度	递归	递推
思考方向	从大往小拆，从后往前	从小往大填，从前往后
实现方式	函数自己调用自己	循环
空间开销	调用栈，$O(n)$	数组，$O(n)$ 或滚动后 $O(1)$
时间效率	可能重复计算，较慢	每项只算一次，较快
代码风格	贴近定义，简洁	需先整理状态与转移
栈溢出	深度大时会溢出	无
适用场景	定义天然递归，或状态不规则	状态转移清晰，需高效

能写出转移方程，优先用递推；递归的分解更直观，可用来梳理思路

记忆化：给递归加缓存

问题：朴素斐波那契递归会重复计算大量子问题。

思路：用一个数组存已算出的结果，下次遇到直接返回。

参考代码

```
long long memo[Maxn + 5];
long long fib(int n)
{
    if (n == 1 || n == 2) return 1;
    if (memo[n] != 0) return memo[n];          // 命中缓存
    return memo[n] = fib(n - 1) + fib(n - 2); // 计算并缓存
}
```

原理：每个 n 只被真正计算一次，后续调用直接返回。

复杂度：时间 $O(n)$，空间 $O(n)$。

限制：`memo` 需初始化为 0，若真实答案可能为 0，需要额外的 `vis` 数组标记。

斐波那契：三种写法对比

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

三种写法

写法	时间	特点
朴素递归	$O(2^n)$	简洁但极慢，$n = 40$ 已明显卡顿
记忆化递归	$O(n)$	保留递归结构，加缓存即可
递推循环	$O(n)$	最快，空间可滚动到 $O(1)$

递推写法

斐波那契：三种写法对比

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
long long a = 1, b = 1, c;
for (int i = 3; i <= n; i++)
{
    c = a + b;
    a = b;
    b = c;
}
cout << b << endl;
```

同一份递推关系，三种实现，效率差异巨大

目录

1 递归基础

2 递推基础

3 对比与进阶

4 总结



总结：两种思路，一套方法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

递归三要素

- 边界条件：最小规模时直接返回
- 递归关系：大问题怎么拆成同类型的小问题
- 状态变化：每次调用必须向边界靠拢

递推四要素

- 状态定义：$dp[i]$ 表示什么
- 转移方程：$dp[i]$ 由哪些前驱推出
- 边界条件：最小规模时的值
- 递推顺序：依赖项必须先算

选择建议

- 定义本身是递归的，先用递归把思路理顺
- 能写出清晰的转移方程，优先用递推
- 递归有重复计算时，加记忆化

附录：常见陷阱速查

错误	修复
递归忘记写边界	补上最小规模的返回值
边界值写错	逐项检查 $n = 1, 2$ 是否正确
状态变化方向错误	保证每次都向边界靠近
递归深度过大	改用递推或加大栈空间
朴素递归超时	加记忆化，或改递推
中间结果 int 溢出	提前用 long long 参与计算
递推数组越界	检查循环上下界与数组尺寸
递推顺序错乱	先确认依赖关系，再确定循环方向

边界要准，方向要真，类型要够